

طراحی الگوریتم‌ها – مرتب‌سازی خطی

Introduction to Algorithm

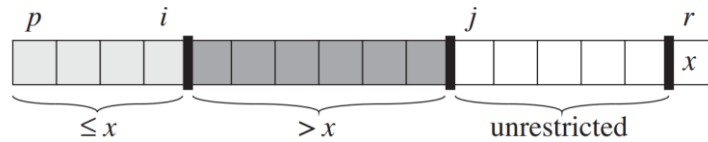
مهدی جوانمردی

مرور جلسه قبل

پیاده سازی PARTITION

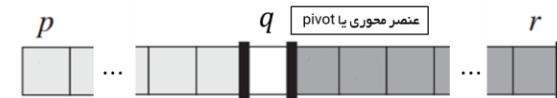
PARTITION(A, p, r)

1 $x = A[r]$ • انتخاب آخرین عنصر به عنوان pivot
2 $i = p - 1$ • تغییر چیدمان عناصر آرایه با رعایت قوانین چهار بخش
3 **for** $j = p$ **to** $r - 1$
4 **if** $A[j] \leq x$ • قرارداد pivot در جایگاه مناسب
5 $i = i + 1$
6 exchange $A[i]$ with $A[j]$
7 exchange $A[i + 1]$ with $A[r]$
8 **return** $i + 1$

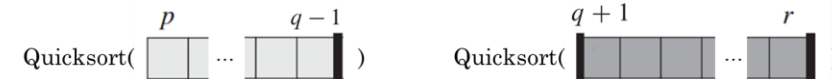


QUICK SORT

Partition (rearrange) the array $A[p..r]$



$$A[p..q-1] \leq A[q] \leq A[q+1..r]$$



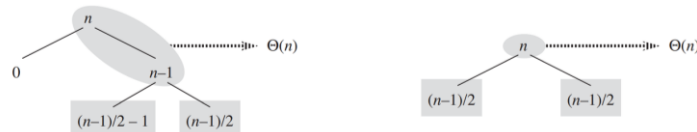
$A[p..r]$ is now sorted

تحلیل زمانی شهودی مرتب سازی سریع

$$\begin{aligned} T(n) &= T(n-1) + T(0) + \Theta(n) \\ &= T(n-1) + \Theta(n) \end{aligned} \quad \longrightarrow \quad \sum_{k=1}^n k = \frac{1}{2}n(n+1) \quad \longrightarrow \quad \Theta(n^2)$$

$$T(n) = 2T(n/2) + \Theta(n), \quad \longrightarrow \quad \text{حالت دوم قضیه اصلی} \quad \longrightarrow \quad T(n) = \Theta(n \lg n)$$

$$T(n) = T(9n/10) + T(n/10) + cn$$



پیاده سازی مرتب سازی سریع

QUICKSORT(A, p, r)

1 **if** $p < r$
2 $q = \text{PARTITION}(A, p, r)$
3 QUICKSORT($A, p, q - 1$)
4 QUICKSORT($A, q + 1, r$)

اولین فراخوانی تابع:

QUICKSORT($A, 1, A.length$).

مرتبسازی مقایسه ای

- الگوریتم‌های معرفی شده تا الان ← مرتبسازی n عدد در $O(n \lg n)$
- مرتبسازی ادغامی و هرمی در بدترین حالت در $O(n \lg n)$
- مرتبسازی سریع در حالت متوسط در $O(n \lg n)$
- خاصیت مشترک الگوریتم‌های معرفی شده تا کنون:

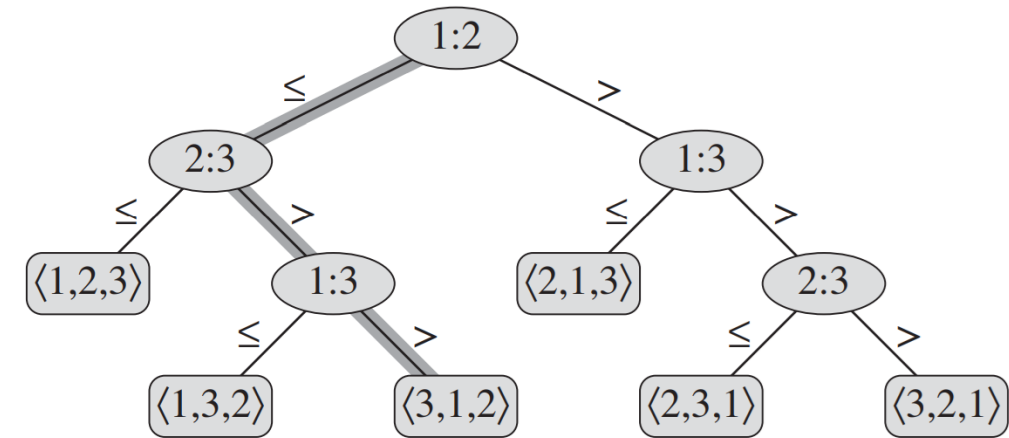
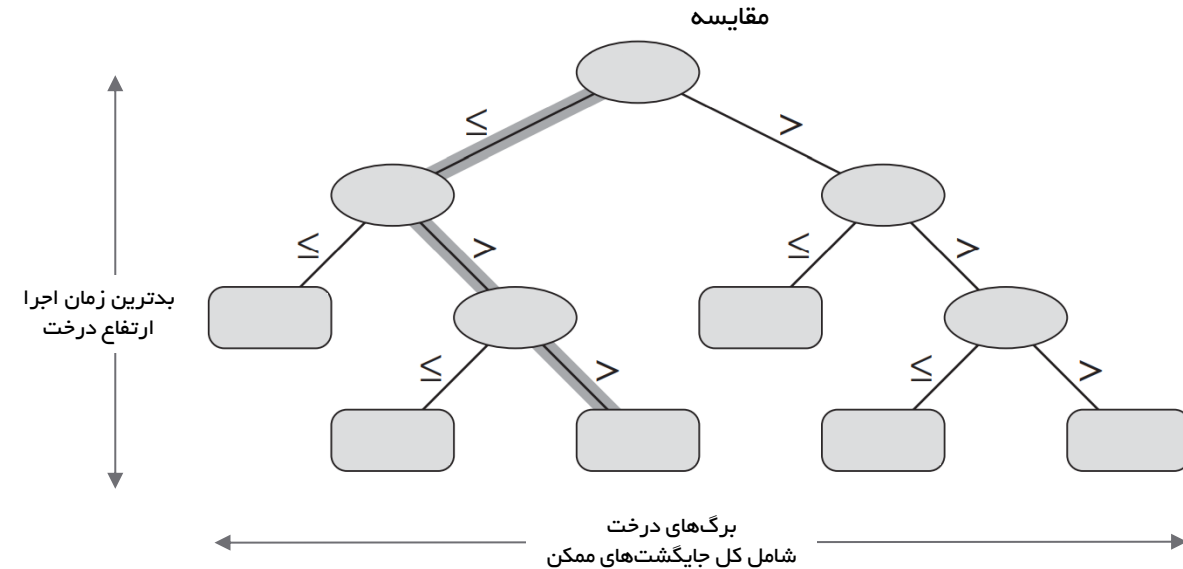


تحلیل رفتار مرتبسازی مقایسه‌ای با استفاده از درخت تصمیم‌گیری دودویی

بهترین بدترین زمان اجرای مرتبسازی مقایسه‌ای

درخت تصمیم‌گیری برای الگوریتم‌های مرتبسازی

درخت تصمیم‌گیری برای مرتبسازی درجی با ۳ عدد



Theorem 8.1

Any comparison sort algorithm requires $\Omega(n \lg n)$ comparisons in the worst case.

تعداد کل جایگشت‌ها n عدد

ماکسیسم برگ برای ارتفاع h

$$n! \leq l \leq 2^h \quad \xrightarrow{\text{لگاریتم از طرفین}} \quad \lg(n!) \leq h = \Omega(n \lg n)$$

تعداد برگ‌ها

Stirling's approximation

$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \left(1 + \Theta\left(\frac{1}{n}\right)\right)$$

مرتب‌سازی خطی

- در صورت وجود برخی اطلاعات در مورد اعداد می‌توان از حد $\Omega(n \lg n)$ عبور کرد

• الگوریتم Counting sort

the input numbers are in the set $\{0, 1, \dots, k\}$

worst-case running time $\Theta(k + n)$

expected running time $\Theta(k + n)$

مرتب‌بیه زمانی

• الگوریتم Radix sort

there are n integers to sort

integer has d digits

digit can take on up to k possible values

worst-case running time $\Theta(d(n + k))$

expected running time $\Theta(d(n + k))$

مرتب‌بیه زمانی

• الگوریتم Bucket sort

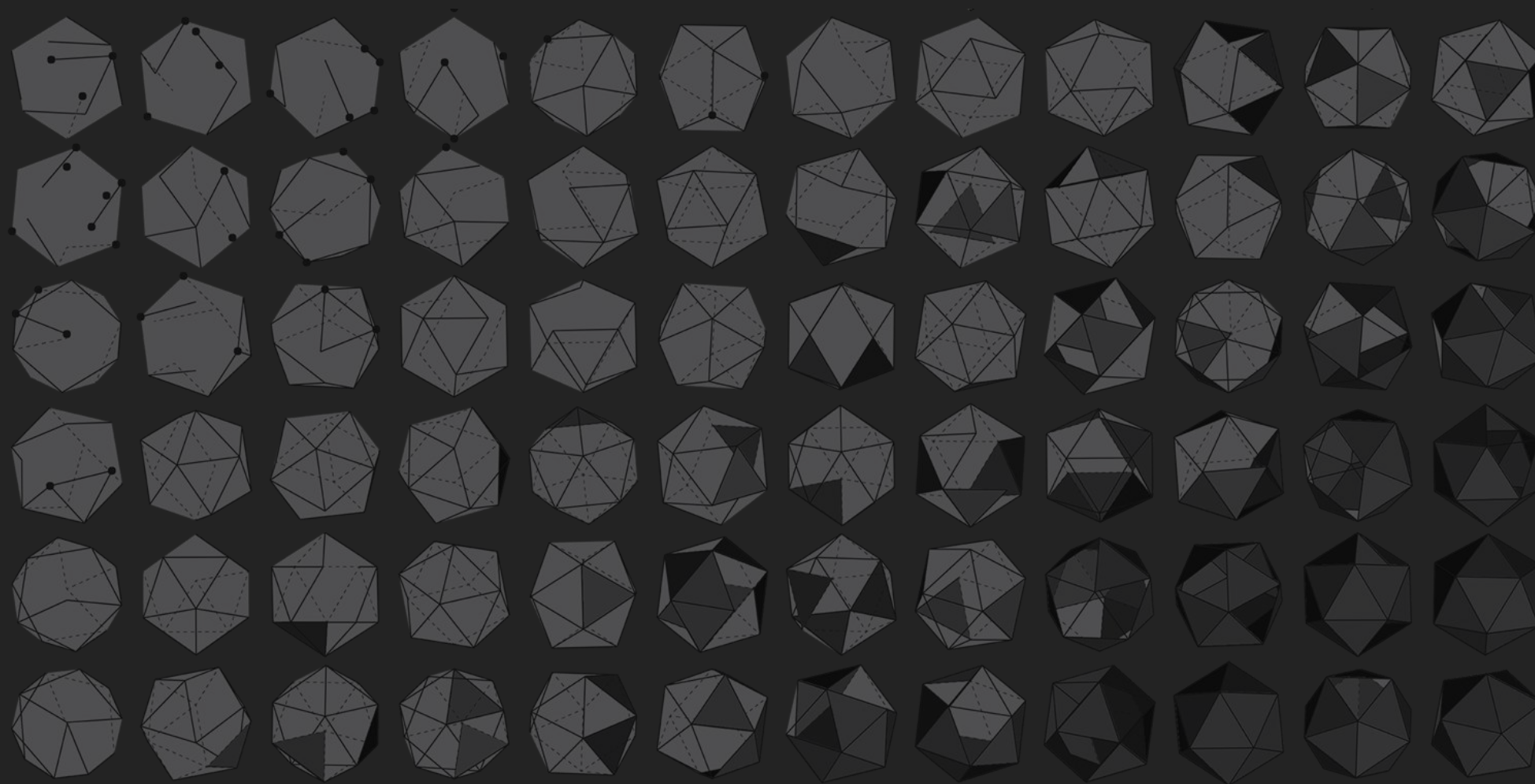
requires knowledge of the probabilistic
distribution of numbers in the input array

real numbers uniformly distributed in the half-open interval $[0, 1)$

worst-case running time $\Theta(n^2)$

average-case running time $\Theta(n)$

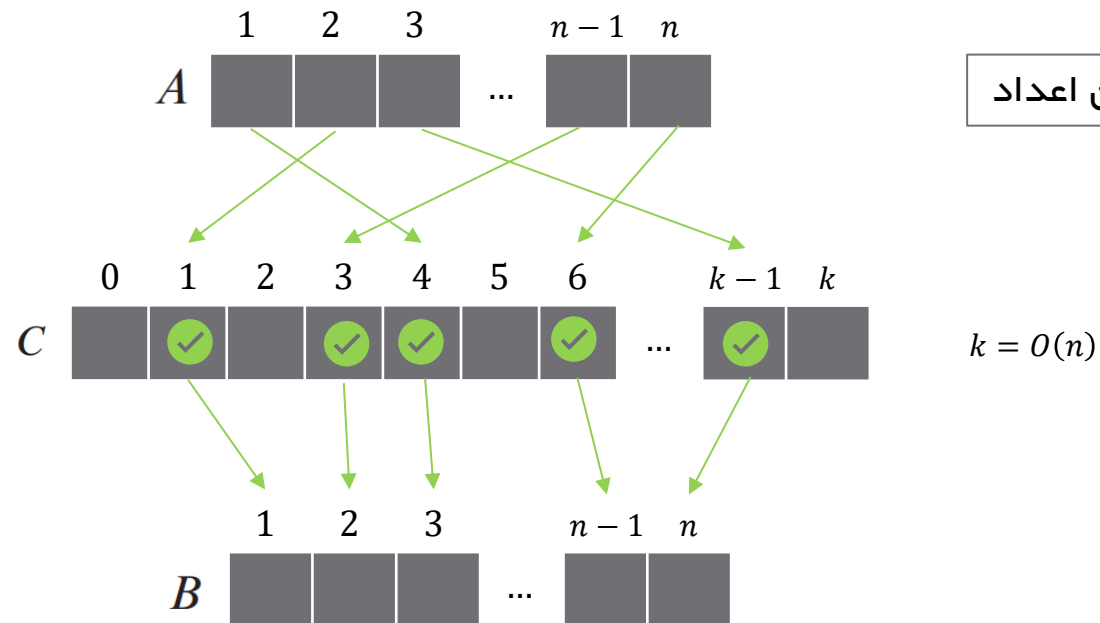
مرتب‌بیه زمانی



مرتب‌سازی شمارشی Counting Sort

مرتب‌سازی شمارشی counting sort

- اطلاعات مازاد: اعداد داخل آرایه اعداد حسابی 0 تا k
- زمان اجرا: در صورتی که $k = O(n)$ باشد زمان اجرای counting sort $\theta(n)$ خواهد بود



در حال کلی و وجود اعداد تکراری؟

- ایده: شمارش تعداد اعداد کوچکتر مساوی هر عدد و قرار دادن آن مستقیماً در جایگاه خود

پیاپی سازی counting sort

COUNTING-SORT(A, B, k)

```

1  let  $C[0..k]$  be a new array
2  for  $i = 0$  to  $k$ 
3       $C[i] = 0$ 
4  for  $j = 1$  to  $A.length$ 
5       $C[A[j]] = C[A[j]] + 1$ 
6  //  $C[i]$  now contains the number of elements equal to  $i$ .
7  for  $i = 1$  to  $k$ 
8       $C[i] = C[i] + C[i - 1]$ 
9  //  $C[i]$  now contains the number of elements less than or equal to  $i$ .
10 for  $j = A.length$  downto 1
11      $B[C[A[j]]] = A[j]$ 
12      $C[A[j]] = C[A[j]] - 1$ 
    
```

$A[1..n]$

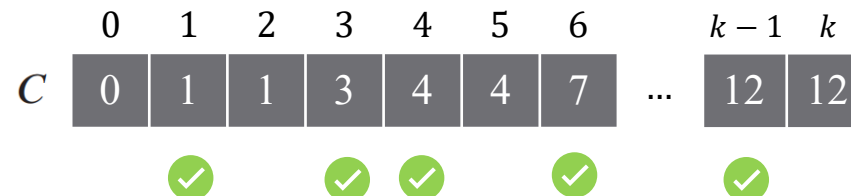
ورودی

$B[1..n]$

مرتب

$C[0..k]$

موقت



مثال counting sort

	1	2	3	4	5	6	7	8
A	2	5	3	0	2	3	0	3

	0	1	2	3	4	5
C	2	0	2	3	0	1

(a)

	0	1	2	3	4	5
C	2	2	4	7	7	8

(b)

	1	2	3	4	5	6	7	8
B							3	

	0	1	2	3	4	5
C	2	2	4	6	7	8

(c)

	1	2	3	4	5	6	7	8
B		0					3	

	0	1	2	3	4	5
C	1	2	4	6	7	8

(d)

	1	2	3	4	5	6	7	8
B		0				3	3	

	0	1	2	3	4	5
C	1	2	4	5	7	8

(e)

	1	2	3	4	5	6	7	8
B	0	0	2	2	3	3	3	5

(f)

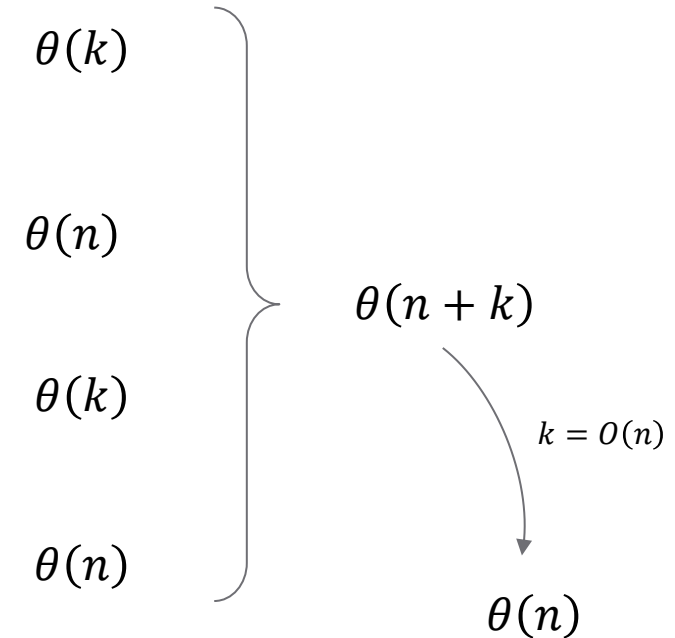
تحلیل زمان اجرای counting sort

COUNTING-SORT(A, B, k)

```

1  let  $C[0..k]$  be a new array
2  for  $i = 0$  to  $k$ 
3       $C[i] = 0$ 
4  for  $j = 1$  to  $A.length$ 
5       $C[A[j]] = C[A[j]] + 1$ 
6  //  $C[i]$  now contains the number of elements equal to  $i$ .
7  for  $i = 1$  to  $k$ 
8       $C[i] = C[i] + C[i - 1]$ 
9  //  $C[i]$  now contains the number of elements less than or equal to  $i$ .
10 for  $j = A.length$  downto 1
11      $B[C[A[j]]] = A[j]$ 
12      $C[A[j]] = C[A[j]] - 1$ 

```

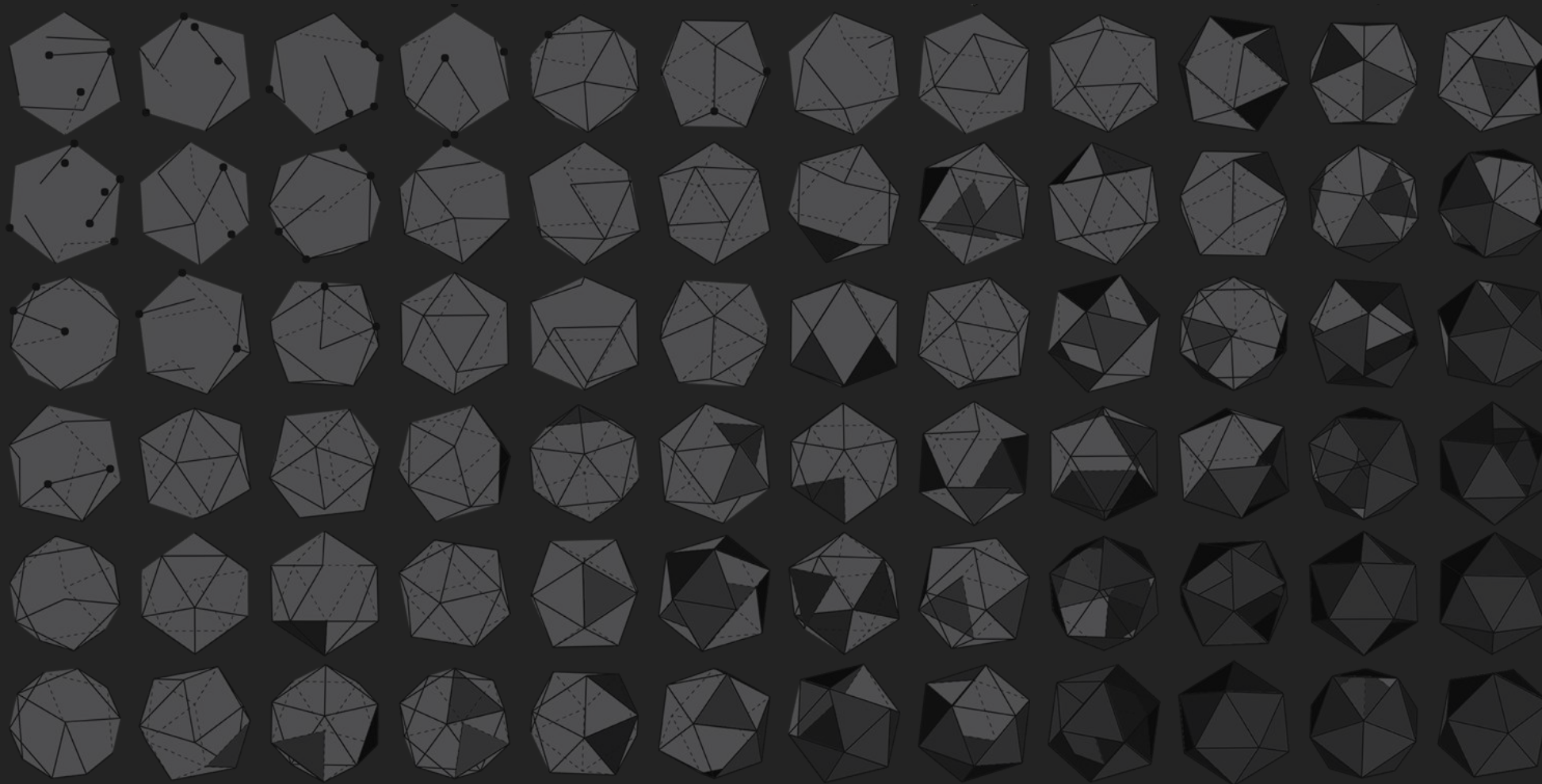


دلیل اینکه زمان اجرای counting sort توانست از حد $\Omega(n \lg n)$ عبور کند
اینست که counting sort یک مرتبسازی مقایسه ای نیست

در عوض counting sort از مقدار خود اعداد بصورت مستقیم برای تعیین محلشان استفاده میکند

An important property of counting sort is that it is *stable*

ویژگی پایداری: ترتیب اعداد برابر در آرایه ورودی و آرایه مرتب شده تغییر پیدا نمی کند

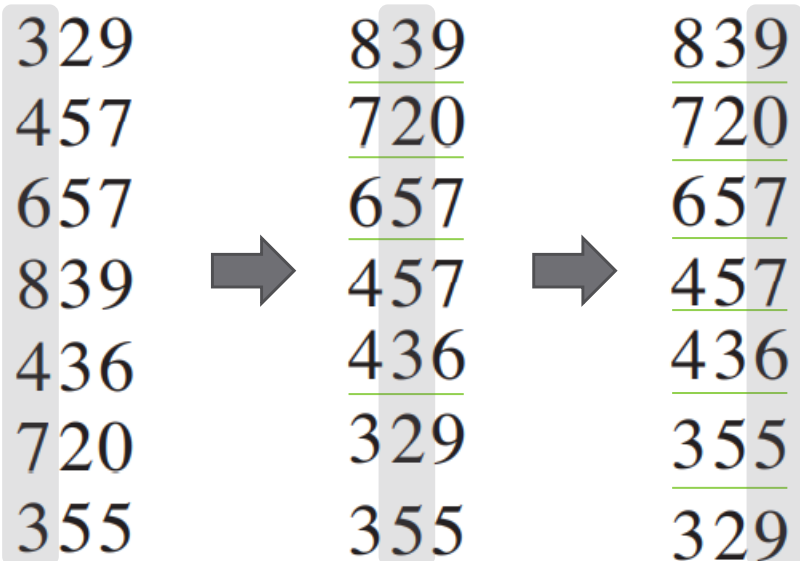


مرتب‌سازی مبنایی Radix Sort

مرتب‌سازی مبنایی radix sort

- مرتب‌سازی ۱۰۰۰ عدد که مقدار هر کدام می‌تواند بین ۰ تا ۹۹۹،۹۹۹ باشد؟
- اطلاعات مازاد: اعداد داخل آرایه دارای d رقم که هر رقم k مقدار متفاوت به خود می‌گیرد
- ایده اصلی: مرتب‌سازی کل از طریق مرتب‌سازی رقم به رقم

روش شهودی – غیر بهینه



- مرتب‌سازی از رقم با ارزش بیشتر
- در بدترین حالت چندبار تابع sort فراخوانی می‌شود؟

d رقم که هر رقم k مقدار متفاوت

مرتب‌سازی مبنایی radix sort

d رقم که هر رقم k مقدار متفاوت

روش شهودی - غیر بهینه

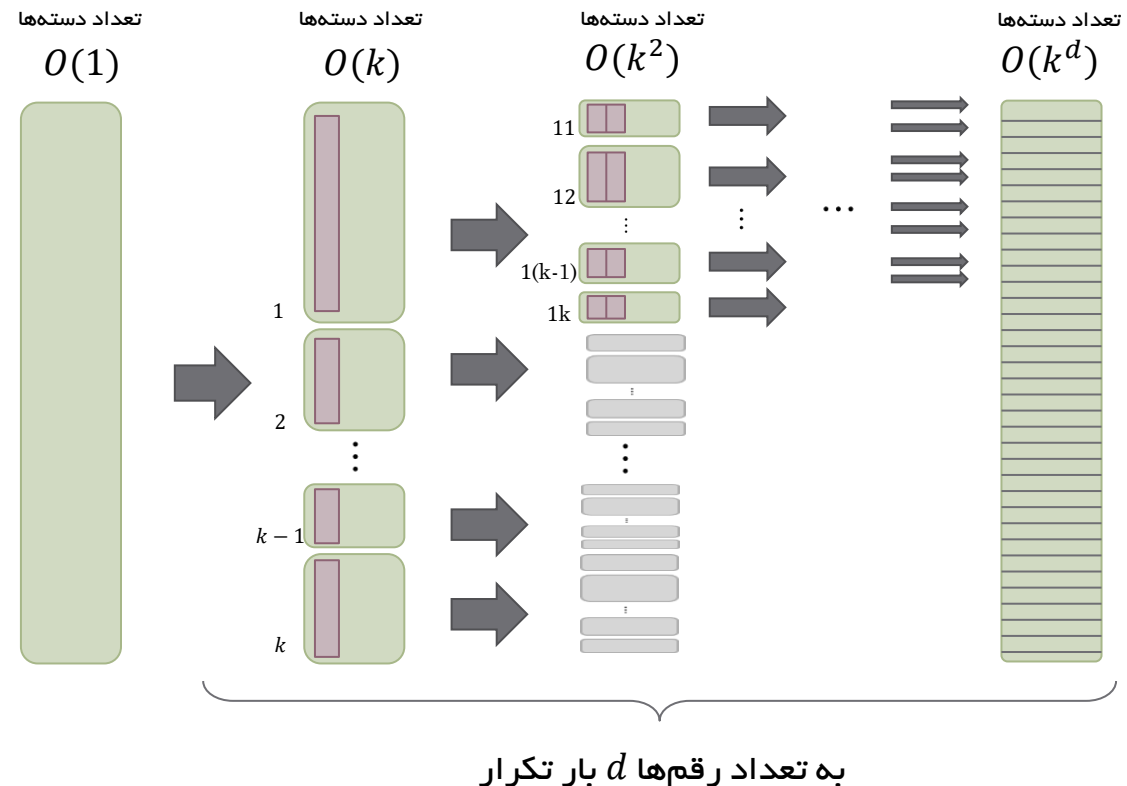
• مرتب‌سازی از رقم با ارزش بیشتر

• در بدترین حالت چندبار تابع sort فراخوانی می‌شود؟ $\Theta(k^{d-1})$

329	839	839
457	720	720
657	657	657
839	457	457
436	436	436
720	329	355
355	355	329

تعداد فراخوان sort در بدترین حالت

$$\leq \sum_{i=0}^{d-1} k^i = \frac{k^d - 1}{k - 1} = \Theta(k^{d-1})$$

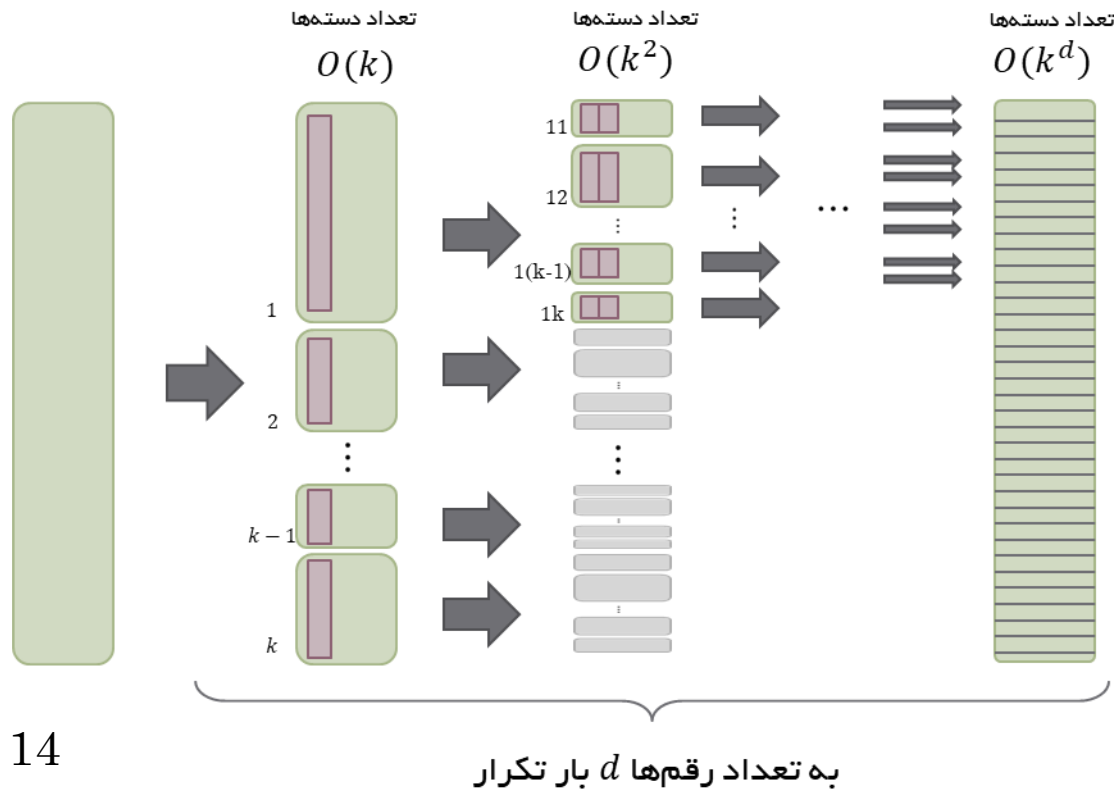


مرتب‌سازی مبنایی radix sort

$$\sum_{i=0}^{d-1} k^i = \frac{k^d - 1}{k - 1} = \Theta(k^{d-1})$$

• در بدترین حالت چندبار تابع sort فراخوانی می‌شود؟

تعداد $n = 16$ عدد $d = 3$ رقمی با $k = 2$ مقدار متفاوت هر رقم



000	000	000	000
001	001	001	000
010	010	000	001
011	011	001	001
100	000	010	010
101	001	011	010
110	010	010	011
111	011	011	011
000	100	100	100
001	101	101	100
010	110	100	101
011	111	101	101
100	100	110	110
101	101	111	110
110	110	110	111
111	111	111	111

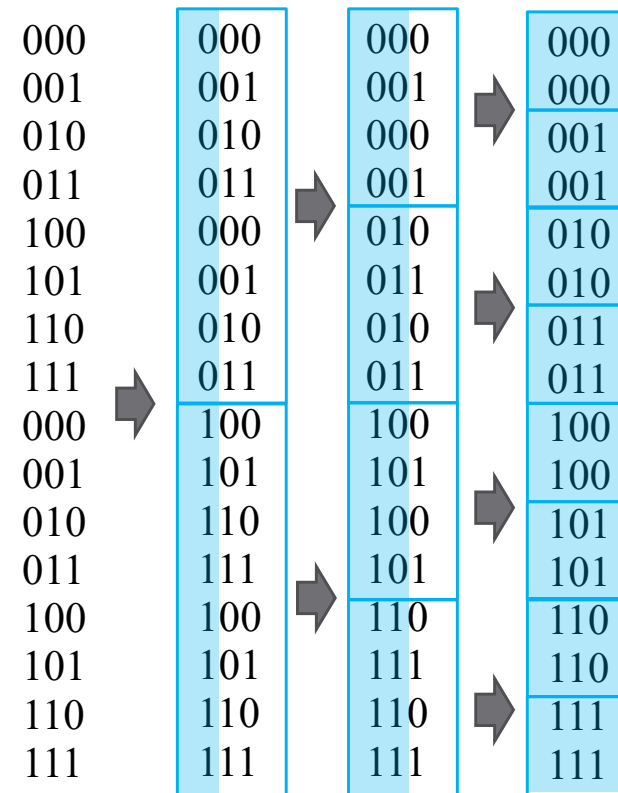
مرتب‌سازی مبنایی radix sort

$$\sum_{i=0}^{d-1} k^i = \frac{k^d - 1}{k - 1} = \Theta(k^{d-1})$$

• در بدترین حالت چندبار تابع sort فراخوانی می‌شود؟

تعداد $n = 16$ عدد $d = 3$ رقمی با $k = 2$ مقدار متفاوت هر رقم

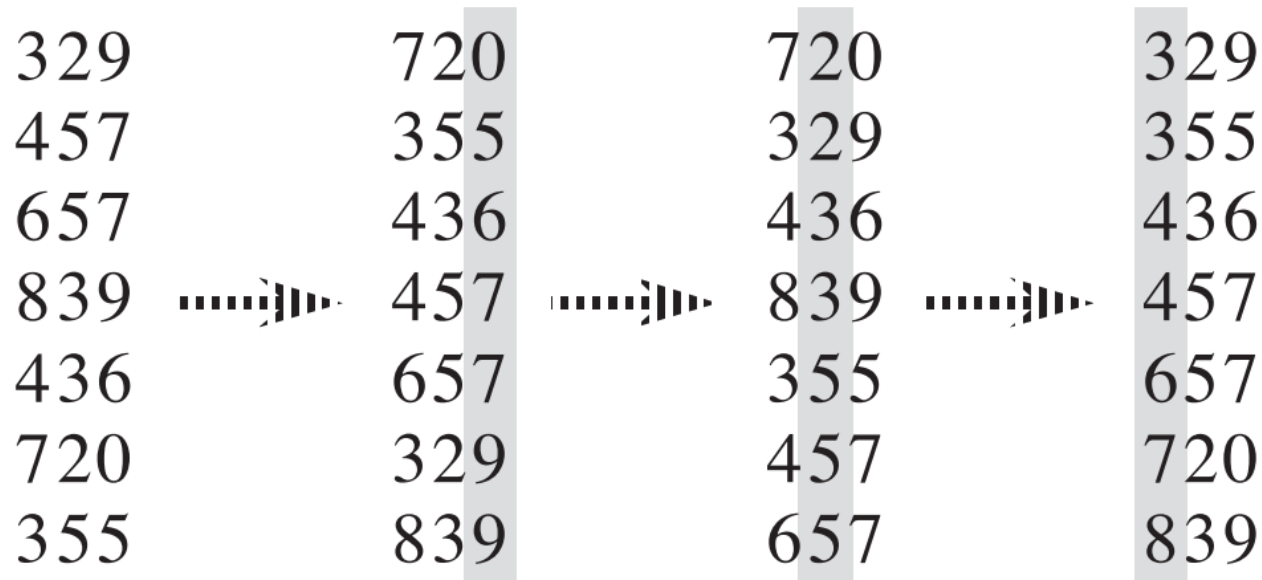
برای حل الگوریتم، می‌بایست در هر لحظه ماکسیمم اطلاعات چند دسته در حافظه نگهداری شود؟



مرتب‌سازی مبنایی radix sort

عکس روش شهودی

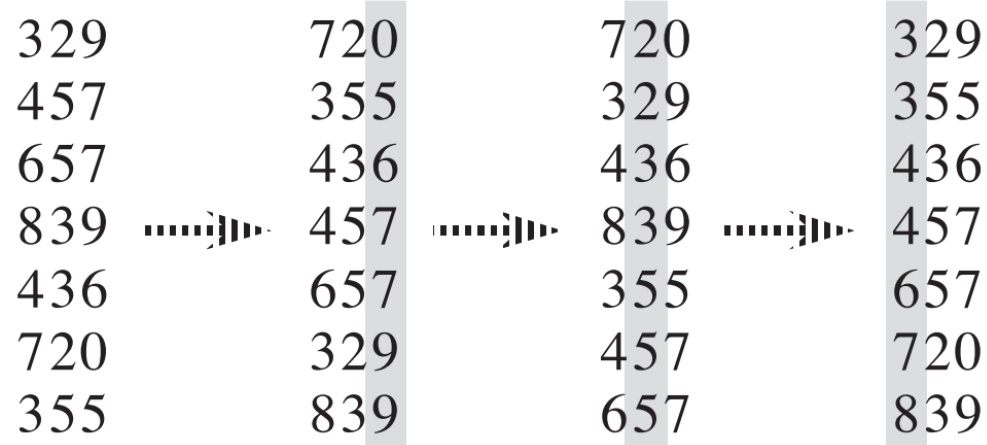
- مرتب‌سازی از رقم با ارزش کمتر
- مرتب‌سازی هر رقم باید پایدار باشد! ← استفاده از counting sort



پیاده‌سازی radix sort و زمان اجرا

RADIX-SORT(A, d)

```
1  for  $i = 1$  to  $d$ 
2      use a stable sort to sort array  $A$  on digit  $i$ 
```



• زمان اجرا: $\theta(d(n + k))$

• اگر $k = O(n)$ و $d = O(1)$ باشد زمان اجرای radix sort $\theta(n)$ خواهد بود

تحلیل دقیق‌تر مرتبه زمانی radix sort

• اگر n عدد b بیتی داشته باشیم، برای هر عدد مثبت دلخواه $r \leq b$ خواهیم داشت:

زمان اجرایی radix sort برای این اعداد $\Theta((b/r)(n + 2^r))$ خواهد بود

به شرطی که مرتب‌سازی پایدار استفاده شده $\Theta(n + k)$ باشد

• اثبات:

برای $r \leq b$ فرض کنید که هر رقم r بیتی باشد و تعداد ارقام برابر $d = \lceil b/r \rceil$
در این صورت هر رقم یک عدد صحیح بین 0 تا $2^r - 1$ خواهد بود
و می‌توان از counting sort با $k = 2^r - 1$ استفاده کرد

$$b = 32, r = 8, k = 2^r - 1 = 255, \text{ and } d = b/r = 4$$



زمان counting sort: $\Theta(n + k) = \Theta(n + 2^r)$

← زمان اجرای radix sort: $\Theta(d(n + 2^r)) = \Theta((b/r)(n + 2^r))$

تحلیل دقیق‌تر مرتبه زمانی radix sort

• اگر n عدد b بیتی داشته باشیم، برای هر عدد مثبت دلخواه $r \leq b$ خواهیم داشت:

زمان اجرایی radix sort برای این اعداد $\Theta((b/r)(n + 2^r))$ خواهد بود

به شرطی که مرتب‌سازی پایدار استفاده شده $\theta(n + k)$ باشد

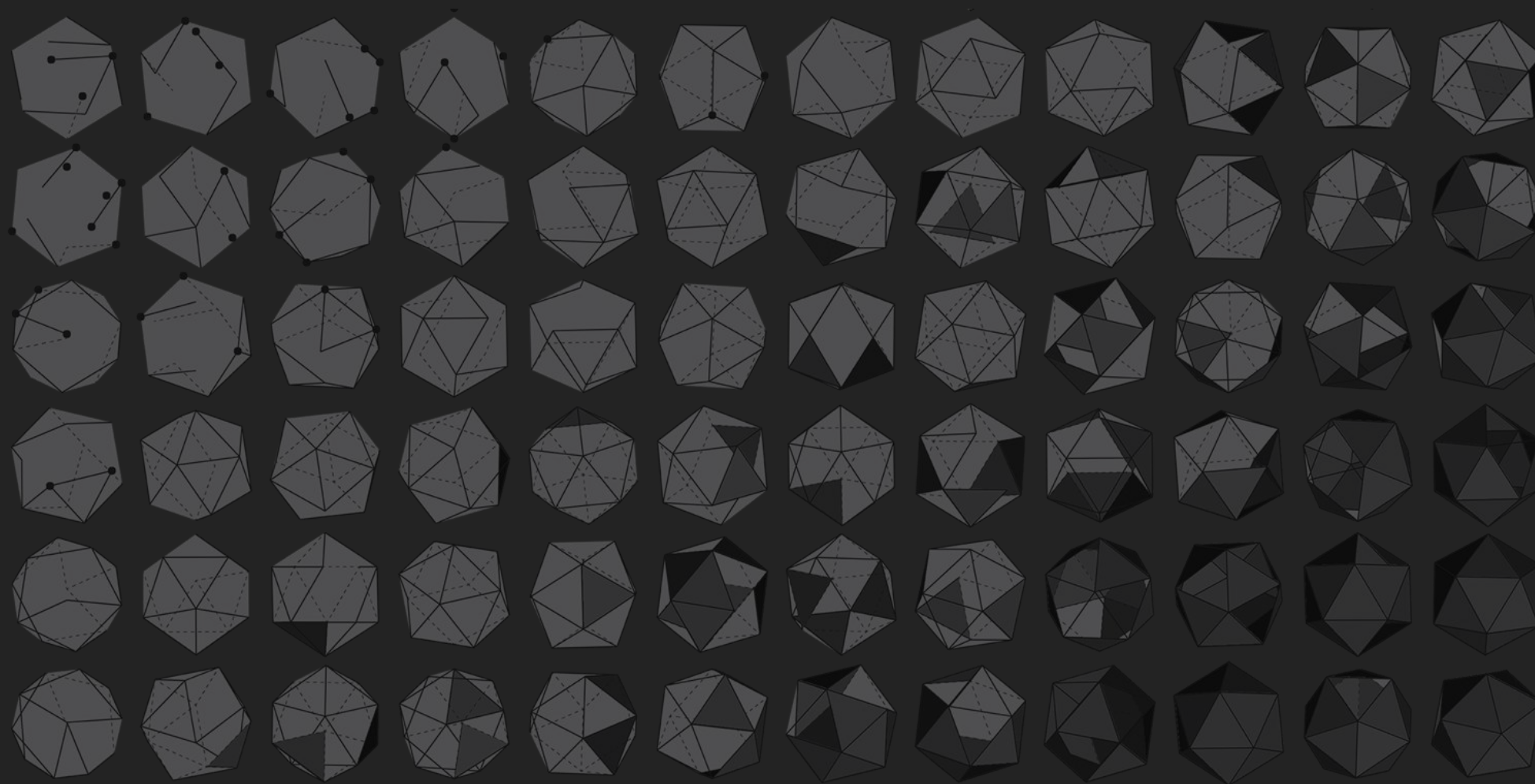
• برای اعداد n و b تعیین $r \leq b$ بگونه‌ای که زمان اجرای $(b/r)(n + 2^r)$ را کمینه کند

حالت اول: $b < \lfloor \lg n \rfloor \leftarrow (n + 2^r) = \Theta(n) \leftarrow$ برای $r = b$ خواهیم داشت: $(b/b)(n + 2^b) = \Theta(n)$

حالت دوم: $b \geq \lfloor \lg n \rfloor \leftarrow$ برای $r = \lfloor \lg n \rfloor$ خواهیم داشت: $(b/\lfloor \lg n \rfloor)(n + n) = \Theta(bn/\lg n)$

radix sort یا انواع مرتب‌سازی مقایسه‌ای؟

- در صورتی که $b = O(\lg n)$ باشد با انتخاب $r = \lfloor \lg n \rfloor$ زمان اجرای radix sort برابر $O(n)$ است.
- در این صورت، تعداد فراخوان‌های radix sort به مراتب کمتر از quick sort خواهد بود در حالی که زمان اجرای هر فراخوان radix sort به مراتب طولانی‌تر از دیگری است.
- اینکه کدام روش مرتب‌سازی بهتر است بستگی به نحوه پیاده‌سازی و سخت‌افزار دارد.
- برای مثال quick sort نسبت به radix از حافظه نهفته بهتر استفاده می‌کند.
- radix sort درجا عمل نمی‌کند (در صورت استفاده از counting sort).
- در حالی که اکثر مرتب‌سازی‌های مقایسه‌ای درجا عمل می‌کنند.



مرتب‌سازی سطلی Bucket Sort

مرتب‌سازی سطلی bucket sort

- اطلاعات مازاد: اعداد آرایه در یک بازه مشخص طی فرایند تصادفی با توزیع یکنواخت تولید شده باشد
- ایده اصلی: مشابه مرتب‌سازی شمارشی اما با آزادی عمل بیشتر

n -element array A

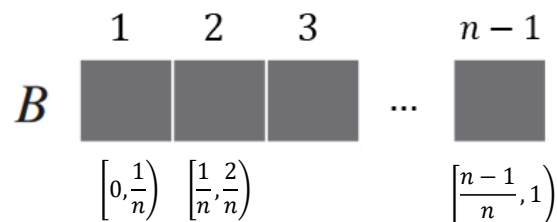
$$0 \leq A[i] < 1$$

توزیع یکنواخت در بازه $[0,1)$



$B[0 \dots n - 1]$

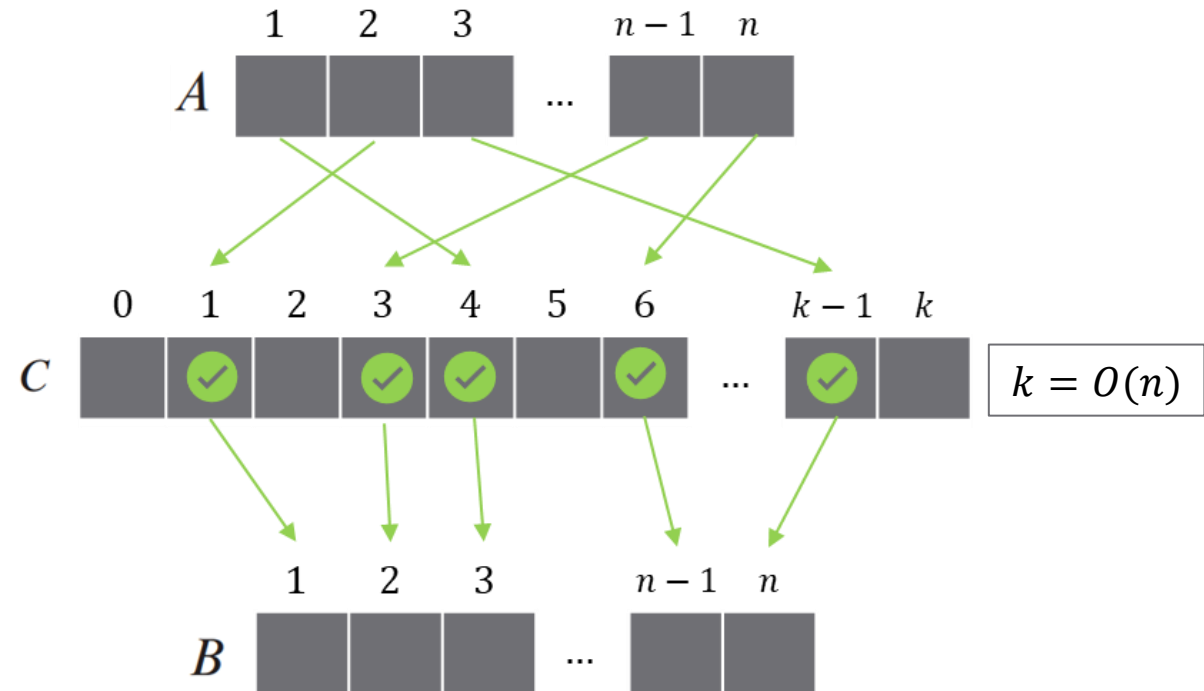
سطل



$A[i]$

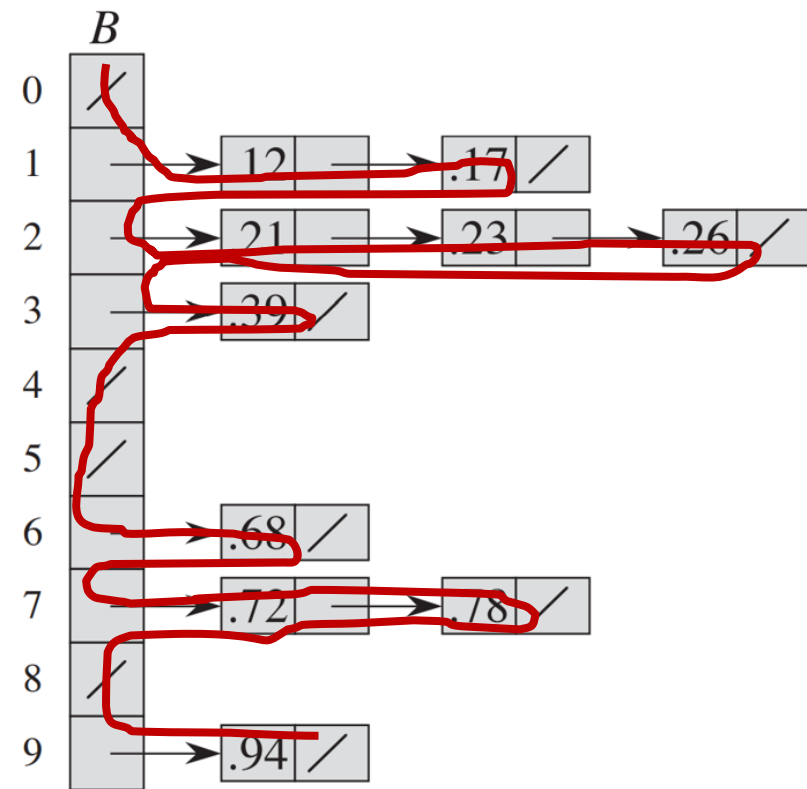
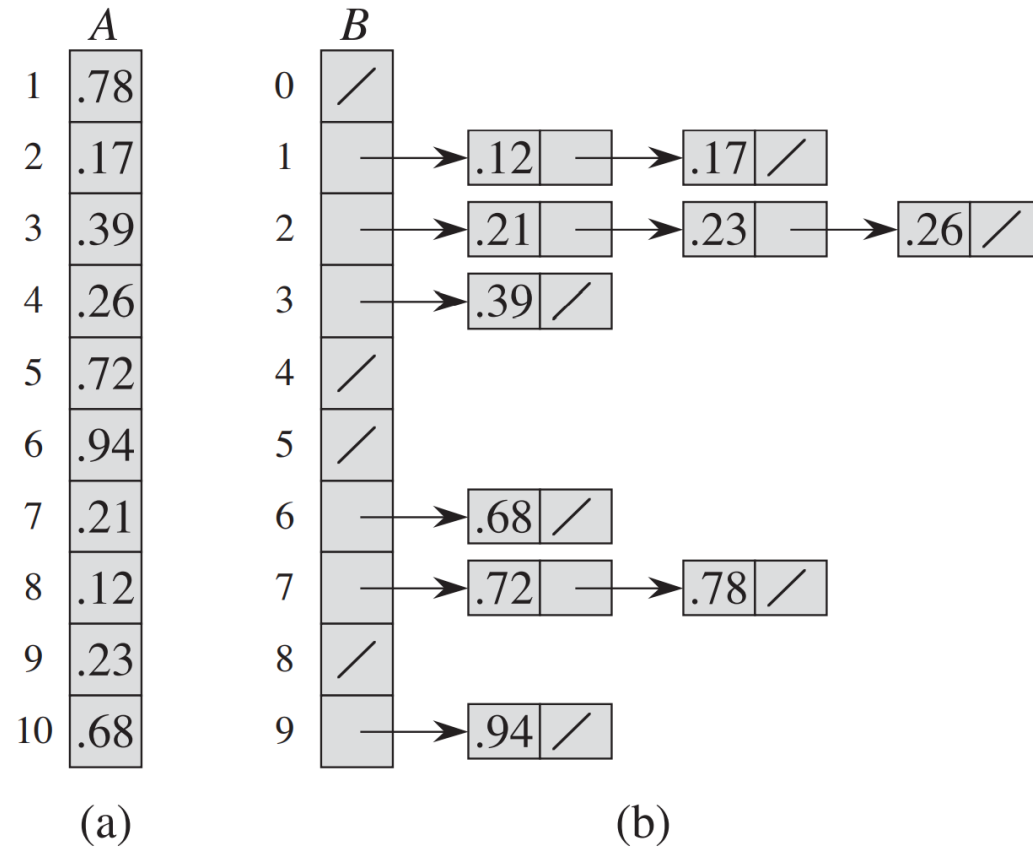


$\lfloor nA[i] \rfloor$



مثال bucket sort

- ساختار آرایه B بصورت linked-list



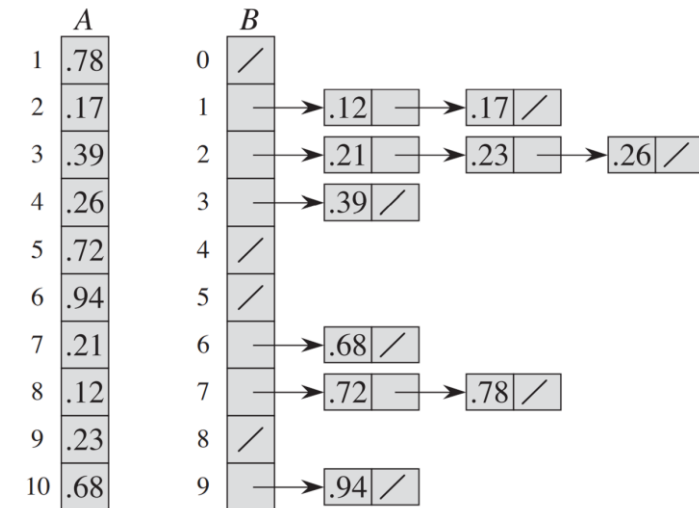
شبهه کد bucket sort

BUCKET-SORT(A)

```

1  let  $B[0..n-1]$  be a new array
2   $n = A.length$ 
3  for  $i = 0$  to  $n-1$ 
4      make  $B[i]$  an empty list
5  for  $i = 1$  to  $n$ 
6      insert  $A[i]$  into list  $B[\lfloor nA[i] \rfloor]$ 
7  for  $i = 0$  to  $n-1$ 
8      sort list  $B[i]$  with insertion sort
9  concatenate the lists  $B[0], B[1], \dots, B[n-1]$  together in order
    
```

• ساختار آرایه B بصورت linked-list



$A[i] \leq A[j] \longrightarrow \lfloor nA[i] \rfloor \leq \lfloor nA[j] \rfloor \longrightarrow$
سطل $A[i]$ یا از سطل $A[j]$ کوچکتر است یا برابر

اگر کوچکتر باشد خط ۹ آن را در جایگاه مناسب قرار می‌دهد

اگر برابر باشد خطوط ۷ الی ۸ آن را مرتب می‌کند

Comparison of sorts

If you only count key comparisons, the leading constants are roughly:

- **Merge sort:**
 $\sim 1.00 \cdot n \cdot \log_2 n$ (tight bounds; near-optimal on comparisons)
- **Quick sort (randomized):**
 $\sim 1.39 \cdot n \cdot \log_2 n$ on average
- **Heap sort:**
 $\sim 2.00 \cdot n \cdot \log_2 n$
- **Insertion sort:**
 $\sim 0.25 \cdot n^2$ on average, $0.5 \cdot n^2$ worst (but $\sim n + \text{inversions}$ if nearly sorted)

تحلیل زمانی bucket sort

- به غیر از خط ۸ شبه شد، همگی با $O(n)$ اجرا می شود
- برای تحلیل زمانی الگوریتم باید مجموع تعداد اجرای خط ۸ محاسبه شود

BUCKET-SORT(A)

```

1  let  $B[0..n-1]$  be a new array
2   $n = A.length$ 
3  for  $i = 0$  to  $n-1$ 
4      make  $B[i]$  an empty list
5  for  $i = 1$  to  $n$ 
6      insert  $A[i]$  into list  $B[\lfloor nA[i] \rfloor]$ 
7  for  $i = 0$  to  $n-1$ 
8      sort list  $B[i]$  with insertion sort
9  concatenate the lists  $B[0], B[1], \dots, B[n-1]$  together in order
```

$$T(n) = \Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)$$

n_i متغیر تصادفی نشانگر تعداد امان‌ها در سطل B_i

تحلیل متوسط زمان اجرای bucket sort

```

7  for  $i = 0$  to  $n - 1$ 
8      sort list  $B[i]$  with insertion sort
    
```

$$T(n) = \Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)$$

n_i متغیر تصادفی نشانگر تعداد المان‌ها در سطل B_i

امیدریاضی و برخی خواص آن

$$E(X) = \sum_{i=1}^n p_i x_i$$

تعریف امیدریاضی برای متغیر تصادفی x_i

$$E(aX + b) = aE(X) + b$$

ویژگی خطی بودن امیدریاضی

Letting $g(x) = ax$, we have for any constant a ,

$$E[aX] = aE[X] .$$

$$\begin{aligned}
 E[T(n)] &= E\left[\Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)\right] \\
 &= \Theta(n) + \sum_{i=0}^{n-1} E[O(n_i^2)] \\
 &= \Theta(n) + \sum_{i=0}^{n-1} O(E[n_i^2])
 \end{aligned}$$

تحلیل متوسط زمان اجرای bucket sort

```

7  for  $i = 0$  to  $n - 1$ 
8      sort list  $B[i]$  with insertion sort
    
```

$$E[T(n)] = \Theta(n) + \sum_{i=0}^{n-1} O(E[n_i^2])$$

ثابت خواهیم کرد:

$$E[n_i^2] = 2 - 1/n \text{ for } i = 0, 1, \dots, n-1$$

تعریف یک متغیر تصادفی جدید:

$$X_{ij} = I\{A[j] \text{ falls in bucket } i\}$$

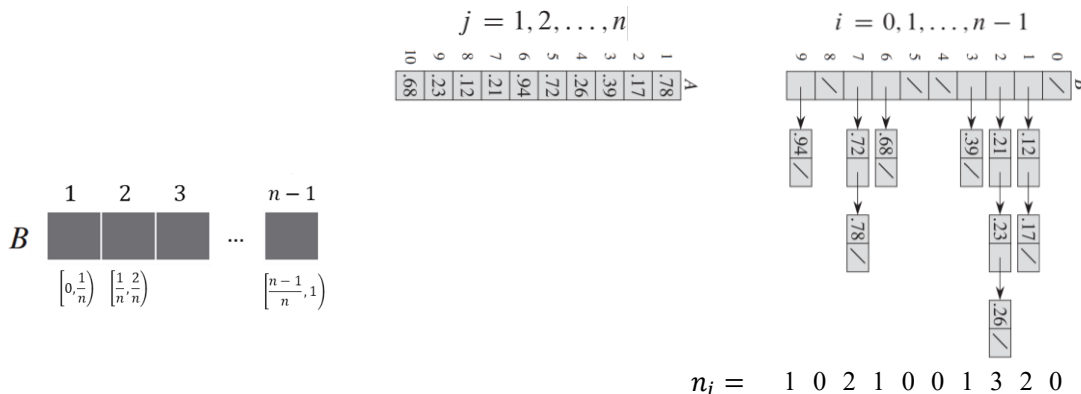
$$n_i = \sum_{j=1}^n X_{ij} \text{ for } i = 0, 1, \dots, n-1 \text{ and } j = 1, 2, \dots, n$$

n_i متغیر تصادفی نشانگر تعداد امان‌ها در سطل B_i

آمار و احتمالات

$$I(A) = \begin{cases} 1, & \text{if } A \text{ happen} \\ 0, & \text{if } A \text{ not happen} \end{cases}$$

متغیر تصادفی



تحلیل متوسط زمان اجرای bucket sort

ثابت خواهیم کرد:

$$\boxed{E[n_i^2] = 2 - 1/n} \text{ for } i = 0, 1, \dots, n-1$$

$$\boxed{n_i = \sum_{j=1}^n X_{ij}}$$

$$E[T(n)] = \Theta(n) + \sum_{i=0}^{n-1} O(E[n_i^2])$$

$$\begin{aligned} E[n_i^2] &= E\left[\left(\sum_{j=1}^n X_{ij}\right)^2\right] \\ &= E\left[\sum_{j=1}^n \sum_{k=1}^n X_{ij} X_{ik}\right] \\ &= E\left[\sum_{j=1}^n X_{ij}^2 + \sum_{1 \leq j \leq n} \sum_{\substack{1 \leq k \leq n \\ k \neq j}} X_{ij} X_{ik}\right] \\ &= \sum_{j=1}^n E[X_{ij}^2] + \sum_{1 \leq j \leq n} \sum_{\substack{1 \leq k \leq n \\ k \neq j}} E[X_{ij} X_{ik}] \end{aligned}$$

تحلیل متوسط زمان اجرای bucket sort

ثابت خواهیم کرد:

$$E[n_i^2] = 2 - 1/n \quad \text{for } i = 0, 1, \dots, n-1$$

$$E[T(n)] = \Theta(n) + \sum_{i=0}^{n-1} O(E[n_i^2])$$

امیدریاضی و برخی خواص آن

$$E(X) = \sum_{i=1}^n p_i x_i$$

تعریف امیدریاضی برای متغیر تصادفی x_i

$$E(XY) = E(X)E(Y) \quad \text{برای متغیر تصادفی مستقل } X \text{ و } Y \text{ داریم:}$$

When $k \neq j$, the variables X_{ij} and X_{ik} are independent

$$E[n_i^2] = \sum_{j=1}^n E[X_{ij}^2] + \sum_{1 \leq j \leq n} \sum_{\substack{1 \leq k \leq n \\ k \neq j}} E[X_{ij} X_{ik}]$$

$$\begin{aligned} E[X_{ij}^2] &= 1^2 \cdot \frac{1}{n} + 0^2 \cdot \left(1 - \frac{1}{n}\right) \\ &= \frac{1}{n} \end{aligned}$$

$$\begin{aligned} E[X_{ij} X_{ik}] &= E[X_{ij}] E[X_{ik}] \\ &= \frac{1}{n} \cdot \frac{1}{n} \\ &= \frac{1}{n^2} \end{aligned}$$

تحلیل متوسط زمان اجرای bucket sort

ثابت خواهیم کرد:

$$\boxed{E[n_i^2] = 2 - 1/n \text{ for } i = 0, 1, \dots, n-1}$$

$$E[T(n)] = \Theta(n) + \sum_{i=0}^{n-1} O(E[n_i^2])$$

$$E[n_i^2] = \sum_{j=1}^n E[X_{ij}^2] + \sum_{1 \leq j \leq n} \sum_{\substack{1 \leq k \leq n \\ k \neq j}} E[X_{ij} X_{ik}]$$

$$\begin{aligned} E[n_i^2] &= \sum_{j=1}^n \frac{1}{n} + \sum_{1 \leq j \leq n} \sum_{\substack{1 \leq k \leq n \\ k \neq j}} \frac{1}{n^2} \\ &= n \cdot \frac{1}{n} + n(n-1) \cdot \frac{1}{n^2} \\ &= 1 + \frac{n-1}{n} \\ &= 2 - \frac{1}{n}, \end{aligned}$$

$$\boxed{E[T(n)] = \Theta(n) + \sum_{i=0}^{n-1} O\left(2 - \frac{1}{n}\right) = \Theta(n)}$$